Project Walkthrough — Hybrid Movie Recommender

A complete, honest explanation of what we built, why, whether it works, and what else we could do. Written against the actual source and the results in `artifacts/metrics/all_metrics.json`.

> **Two things to fix before submission (flagged up front):**
>
> 1. The **Stacked Hybrid is missing from `all_metrics.json`** — notebook 03 wasn't fully re-run after that model was fixed (only 7 of 8 models are in the results).
> 2. The **ranking (F1@K) numbers have a tie-ordering artifact** that makes a constant predictor "win" — fixable with a candidate shuffle. See §7.

---

1. The problem we're solving

Assignment Θέμα 2: build a **hybrid recommender** that fuses a **content-based** method with a **collaborative-filtering** method, then prove it beats each one alone on RMSE, MAE, Precision@K, Recall@K, F1@K — and wrap it in an app.

A recommender answers: *given what user u has rated, what rating would they give item j, and which unseen items should we show them?* The two classic families:

- **Collaborative Filtering (CF)** — "people who rated like you also liked X." Uses only the rating matrix. Powerful with lots of ratings; useless for brand-new items/users (cold start).
- **Content-Based (CB)** — "you liked these, here are similar *items by their attributes*." Uses item metadata (genres, tags). Handles cold-start items, but over-specializes and ignores cross-user taste patterns.

Hybrid = combine them to get CF's accuracy *and* CB's cold-start robustness. That is the whole point of the assignment.

---

2. The data — MovieLens 25M

Ratings	25,000,095
Users	162,541
Movies	62,423
Scale	0.5–5.0 (half-star)
Tag applications	~1.09M
Sparsity	**> 99.8%**

Six raw CSVs: `ratings`, `movies`, `tags`, `genome-scores`, `genome-tags`, `links`.

Key EDA findings (notebook 01):

- **Positivity bias** — modal rating is 4.0, low ratings rare → justifies the **relevance threshold ≥ 4.0** for "did the user like it."
- **Power-law / long tail** — most users rate few films, most films get few ratings → sparsity is the core CF challenge and the reason CB helps.
- **Genre dominance** — Drama / Comedy / Thriller dominate.
- **Temporal trend** — rating volume grows over time → a **temporal split** is the honest protocol (a random split would leak the future into training).

> ⚠ The **tag genome** (1,128 relevance scores/movie) is profiled in EDA but **never used as a model feature** — content features come from free-text tags only. That is unused signal (see §8).

---

3. The three notebooks

- **`01_eda.ipynb` — Explore & split.** Loads CSVs, profiles everything above, then produces the **user-wise temporal 80/10/10 split**: each user's ratings sorted by time, first 80% → train, next 10% → val, last 10% → test. Users with < 5 total ratings dropped. Saves processed parquets + splits — the foundation everything else loads identically.
- **`02_features.ipynb` — Build item features.** Turns each movie into a vector (§4). Saves the feature matrix + fitted transformers so serving never re-fits.
- **`03_train_evaluate.ipynb` — Train all 8 models & evaluate once.** The heart of the project: trains every model, evaluates exactly once on the untouched test set under a leak-free protocol, writes `all_metrics.json`, saves all models for the app.

---

4. The features (content representation)

Each movie becomes a **276-dimensional vector** = two blocks stacked:

1. **Genre block (~20 dims)** — multi-hot indicator (`Action=1, Comedy=0, …`). Interpretable, zero-cost.
2. **Text block (256 dims)** — TF-IDF over `clean_title + aggregated_tags` (bigrams, `min_df=5`, sublinear TF), compressed with **TruncatedSVD → 256 dims** (Latent Semantic Analysis: turns a huge sparse vocabulary into a dense semantic embedding).

Similarity between two movies = **cosine similarity** of these vectors.

---

5. The models

We built **8** (the assignment requires 3: one CB, one CF, one hybrid — so this is well over-spec):

#	Model	Family	One-liner
1	Global Mean	baseline	predict the training average for everything
2	Popularity	baseline	score by how many ratings an item has
3	**Content-Based**	CB	rate j by your ratings on its content-nearest neighbors
4	User-kNN	CF	"users like you rated j as…"
5	Item-kNN	CF	"items like j that you rated…"
6	**SVD**	CF	matrix factorization with bias terms
7	**Weighted Hybrid**	hybrid	$\alpha \cdot \text{SVD} + (1-\alpha) \cdot \text{CB}$
8	**Stacked Hybrid**	hybrid	Ridge meta-learner over all base predictions

Content-Based — predicts via a *mean-centred weighted average*: $\hat{r}(u,j) = \bar{r}_u + \Sigma \text{sim}(i,j) \cdot (r_{u,i} - \bar{r}_u) / \Sigma|\text{sim}|$, over the top-50 content-similar items you've rated. (Engineering note: it densifies + L2-normalizes the matrix once so cosine is a single fast BLAS matmul, with an LRU cache.)

User-kNN / Item-kNN — Surprise `KNNWithMeans`, Pearson-baseline similarity, k=80. To fit in RAM, Item-kNN keeps only the 15K most-rated movies and User-kNN samples 20K users (full similarity matrices would be ~15 GB / ~98 GB).

SVD — learns latent user/item factors + biases: $\hat{r} = \mu + b_u + b_i + q_i^T p_u$. Grid-searched by 5-fold CV.

How the hybrid combines the two (the part the report must explain)

- **Weighted Hybrid:** linear blend $\alpha \cdot \text{SVD} + (1-\alpha) \cdot \text{CB}$, α found by sweeping $0 \to 1$ on the validation set. If CB can't score an item (cold start), falls back to SVD. Simple, interpretable.
- **Stacked Hybrid:** a **Ridge meta-learner** that *learns from data* how to weight `[content, user-kNN, item-kNN, SVD]` predictions plus side features (popularity, rating counts). Trained on **5-fold out-of-fold** predictions so the meta-model never sees a base model predicting on its own training data (no leakage). More powerful, less interpretable.

---

6. Evaluation protocol

- **Rating metrics (RMSE, MAE):** over the full test set.
- **Ranking metrics (P/R/F1@K):** for 1,000 sampled test users, rank each user's relevant items against **100 random "negative" items**, $K \in \{5, 10, 20\}$, relevance $\geq$ 4.0 (standard NCF/BPR-style sampled-negatives protocol).
- Hyperparameters tuned only on val/CV; test touched once.

---

7. Is it good? — the honest assessment

Rating accuracy (RMSE/MAE) — □ clean, sensible, publishable

Model	RMSE	MAE
Weighted Hybrid	**0.8096**	**0.6074**
SVD	0.8109	0.608
Item-kNN	0.8336	0.6223
User-kNN	1.0401	0.8119
Content-Based	1.0462	0.7831
Global Mean	1.0609	0.8339
Popularity	2.712	2.4856

Textbook and defensible: **SVD dominates**, the **Weighted Hybrid edges it out marginally** (0.8096 vs 0.8109), kNN and CB are mid, naive baselines worst. Good report story: *the hybrid is at least as good as the best single model.*

Caveat: the hybrid beats SVD by **0.0013 RMSE** — basically nothing — because α tuned almost entirely toward SVD (the CB signal is weak/noisy), so the "hybrid" is ~95% SVD. Honest framing: *the hybrid doesn't hurt and adds cold-start coverage, but on this data CF alone already captures most of the signal.*

Ranking (F1@K) — ⚠ misleading as computed

Model	F1@10
Global Mean	0.5806
User-kNN	0.5539
Popularity	0.52
Content-Based	0.4696
Item-kNN	0.3702
Weighted Hybrid	0.2967

Model	F1@10
SVD	0.2947

This looks **backwards** — a constant predictor "wins" and the best-RMSE models rank worst. Two reasons, one of which is a bug:

1. **Tie-ordering artifact (genuine bug).** In `evaluate_ranking_sampled`, candidates are built as `relevant_items + negatives` — relevant ones first. Python's sort is *stable*, so when a model outputs **identical scores** (Global Mean gives everything ~3.5; User-kNN/CB fall back to user-mean constantly), ties break by original order → **relevant items float to the top for free**, inflating P/R/F1. Models with genuinely varied scores (SVD) get no such gift, so their "honest" ranking looks worse. **Fix:** shuffle candidates before sorting, or add a random tie-break.
2. **A real phenomenon worth stating anyway:** RMSE-optimal ≠ ranking-optimal. SVD minimizes squared error but isn't trained to *push the few relevant items above 100 randoms*. A classic result — motivates ranking-first methods (BPR, learning-to-rank, GNNs).

Action items before submission

1. **Re-run notebook 03** — the Stacked Hybrid is absent from `all_metrics.json` (7 of 8 models shown).
2. **Fix the ranking tie-break** (shuffle candidates) and regenerate — otherwise a grader will immediately ask why Global Mean "wins" ranking.
3. Consider reporting **NDCG** or **AUC** alongside F1@K — standard and less brittle to ties.

Overall verdict

The *engineering* is genuinely strong (clean library, leak-free protocol, memory-aware CF, two hybrid strategies, a working app — well beyond the minimum). The *rating-accuracy* results are solid and tell a clean story. The *ranking* evaluation has a fixable bug that currently makes its headline numbers indefensible. Fix that and re-run, and this is a very good submission.

---

8. What other approaches exist — graph / Neo4j / etc.

The repo is named `knowledge_graphs_ass`, so this is a natural question. Note: graph methods aren't *required* by Θέμα 2 (they're closer to Θέμα 1, link prediction on a bipartite graph) — treat these as **extensions / alternatives**, not gaps.

Graph-based recommendation

Model the data as a graph: `(User)-[RATED]->(Movie)-[HAS_GENRE]->(Genre)`, `(Movie)-[TAGGED]->(Tag)`. Recommendation becomes **link prediction** on the user–item bipartite graph:

- **Heuristics:** Common Neighbors, Jaccard, Adamic-Adar (exactly Θέμα 1's option 2).
- **Node embeddings:** Node2Vec / DeepWalk → train a classifier on node-pair features.
- **Graph Neural Networks (current SOTA for CF): LightGCN**, NGCF, PinSage, GraphSAGE — learn user/item embeddings by message-passing over the interaction graph and optimize a *ranking* loss (BPR). These typically **beat plain matrix factorization on ranking metrics** — exactly where this pipeline is weak. The single highest-impact upgrade for better Precision/Recall@K.

Neo4j specifically

Neo4j is a graph database with a **Graph Data Science (GDS)** library shipping recommender-relevant algorithms:

- Store the data as a property graph (users, movies, genres, tags as nodes).
- **Node Similarity** (Jaccard/cosine over neighborhoods) → item-item or user-user CF directly in Cypher.
- **Link Prediction ML pipelines** — GDS has a built-in pipeline: generate `FastRP`/`GraphSAGE`/`Node2Vec` embeddings, train a link predictor, evaluate. Essentially Θέμα 1 turnkey.
- **PageRank / personalized PageRank** for popularity- and proximity-based recs.

Knowledge Graph Embeddings (KGE)

Embed a richer movie knowledge graph (genres, tags, actors, directors via `links.csv` → IMDb/TMDb) with **TransE / DistMult / ComplEx**, then score `(user, likes, movie)` triples. Content + structure fused — a "knowledge-graph hybrid," very on-theme for the course name.

Non-graph improvements (smaller effort, on-topic for Θέμα 2)

- **Use the tag genome** — it's loaded but unused. 1,128 dense relevance scores/movie would give a much richer content signal than free-text tags, likely making CB (and thus the hybrid) genuinely contribute instead of being drowned by SVD.
- **Optimize for ranking, not RMSE** — switch SVD → **BPR** (Bayesian Personalized Ranking) or add a learning-to-rank meta-model. Directly addresses the poor F1.
- **Neural CF / autoencoders** — NCF, Mult-VAE, or Factorization Machines (libFM) as extra CF bases.
- **Switching hybrid** — route to CB when the item is cold (few ratings) and to CF when it's well-observed, instead of a fixed blend.

---

9. Suggested next steps (priority order)

1. **(correctness)** Fix the ranking tie-break (shuffle candidates) and re-run notebook 03 so the

table is honest and includes the Stacked Hybrid.

2. **(lift)** Wire the tag genome into the content features for a real hybrid contribution.
3. **(extension)** Prototype a graph/Neo4j or LightGCN approach for a ranking-optimized comparison.